

StormBonk

AI Design & Development Report

Survive the Optimization

Miguel Requena López • Marcos González Palazón • Amine Saghrouchni Yousfi
Group 07 Games

2026

“You are the outlier. The data point that refuses to be normalized. The Algorithm does not tire. It does not run out of constructs. It optimizes, over and over, until it finds the configuration you are not ready for.”

— *StormBonk Lore*

Abstract. This document is the final development report for *StormBonk*, a 2D top-down wave-survival roguelite developed in UNITY. The project’s main goal was never to ship a polished commercial product, but to use a small, focused game as a sandbox to *design* game AI — not just implement it. We stack four classic techniques (**Fuzzy Logic**, **A***, **Hierarchical FSM**, **Blackboard**) on top of a deliberately simple combat loop, and let them collide on screen. The result is enemies that don’t just chase the player, they *feel like they have intentions*. This report walks through the game itself, every AI subsystem, why we picked each technique for each role, the design trade-offs we made along the way, and what we ended up cutting.

Contents

1	Introduction	3
1.1	Why this project, and why this report	3
1.2	The pitch in three sentences	3
1.3	Genre, scope, platform	3
1.4	What changed from the GDD	3
2	High-level design philosophy	3
3	Game overview: the player loop	4
3.1	The 30-second loop	4
3.2	Why no attack button?	5
4	The player character	5
4.1	Stats and progression	5
4.2	Auto-aim fireball	5
4.3	Arcane Pulse (automatic defensive AoE)	6
5	Enemy archetype 1: Melee Hunter (Fuzzy Logic)	6
5.1	Role	6
5.2	Why Fuzzy Logic for this enemy	6
5.3	Membership functions	7
5.4	Rule base (Mamdani-style)	7
5.5	Bonus: Blackboard read	7

6	Enemy archetype 2: Charger (Hierarchical FSM + Blackboard)	8
6.1	Role	8
6.2	Why an HFSM (and not a flat one)?	8
6.3	Why telegraph the charge with a 1-second pause?	8
6.4	Why the Charger takes $\times 2$ damage while Stunned	9
6.5	Coordination via Blackboard	9
6.6	Stun handshake with the player	9
7	Enemy archetype 3: Mage / Ranged Shooter (FSM)	9
7.1	Role	9
7.2	The three-state FSM	10
7.3	Why the Mage shoots while retreating	10
7.4	The three-band distance system	10
8	Pathfinding: a custom A*	10
8.1	Why we wrote our own A* instead of using NavMesh	10
8.2	Cost function and heuristic	11
8.3	Why every enemy queries A*, every 0.3–0.5 seconds	11
9	The Blackboard: small, static, surprisingly load-bearing	11
9.1	Why a static class instead of a singleton	11
9.2	What the Blackboard actually holds	12
9.3	Emergent coordination	12
10	Spawning: where do enemies appear, and why	12
11	Wave / level progression	13
11.1	Structure	13
11.2	Why two different curves?	13
11.3	When the player gets stronger	13
12	UI / UX	13
13	What we cut, and why	14
14	Reflection & takeaways	14
14.1	What worked	14
14.2	What we'd do differently	15
14.3	The thesis, one more time	15
A	File map	15
B	Tuning constants (current build)	16

1 Introduction

1.1 Why this project, and why this report

The temptation when you are asked to make “a game with AI” is to pick the fanciest technique you’ve heard of (neural nets, behavior trees with 50 nodes, GOAP, you name it) and brute-force it onto something. We tried to do the opposite. The slides we worked from kept hammering one idea: **complexity is a lie**. A simple algorithm, used with intent and covered by good animation, will look smarter than a 2000-line behavior tree that nobody can debug. *Pac-Man* is the canonical example: the four ghosts share *the same movement code* and only differ in their target tile, yet players still talk about “the orange one being shy” or “the red one being aggressive.” The intelligence lives in the *design*, not in the algorithm.

So this report is structured around **design rationale** first and implementation second. Every time we use Fuzzy Logic, A*, an HFSM or the Blackboard, the question we try to answer is not “how do we code this” but “what does this make the player *feel*, and is that the feeling we want?”

1.2 The pitch in three sentences

You are **Storm Breaker**, the last unnormalized data point in a universe that an omniscient algorithm has compressed into perfect, sterile order. Constructs spawn from the edges and walk you down with three different brains. You auto-fire, you reposition, you survive — or you don’t, and the algorithm wins this round.

1.3 Genre, scope, platform

- **Genre.** 2D top-down action roguelite, wave-survival. Closer to *Vampire Survivors* than to *Hades*: short runs, a single screen, attention budget split between movement and reading enemy patterns.
- **Engine.** Unity (2D), C# scripts.
- **Platform.** PC (Windows). Solo, keyboard-only.
- **Run length.** A run lasts as long as you stay alive — typically 5–10 minutes.

1.4 What changed from the GDD

The original Game Design Document was an ambitious wishlist (12 stages, elemental “Tomes”, dashes, sprint, three biomes, bosses, meta-currency, shrines, etc.). We made aggressive cuts: the final game is one arena, no elemental Tome system, no dash, no manual aiming, and a 3-level / 5-wave progression instead of 12 stages. We kept what carried the AI experiment and dropped what didn’t. The next sections describe what *actually exists*, not what was planned.

2 High-level design philosophy

Three rules of thumb governed almost every decision we made:

(1) **Design before code.** Each enemy was a *role* before it was a script. The Hunter is the “adaptive pursuer”, the Charger is the “shock weapon”, the Mage is the “cold sniper.” Once we had three clear archetypes, the AI technique to implement each one was almost self-suggesting — and the team stopped arguing about it.

(2) Believability > Optimality. An enemy that picks the mathematically optimal route to your face every frame doesn't feel intelligent, it feels like an aimbot. We deliberately introduced hesitation (the Charger's **Prepare** state, the Mage's **Retreat**, the Hunter's **Cauteloso** mode when wounded) because that's what humans read as *thinking*.

(3) Emergent before scripted. The game has no scripted encounters. Every wave is a roll of the dice on which enemies are present and where they spawn. The interesting moments — a Charger stunning you while two Hunters rush in — are not authored, they emerge from the rules. The *Blackboard* is the glue that lets that happen without coupling enemies to each other.

The diagram below summarizes the AI stack. Four independent techniques run at the same time, each owned by the agent that benefits most from it; the Blackboard is the only thing they all see.

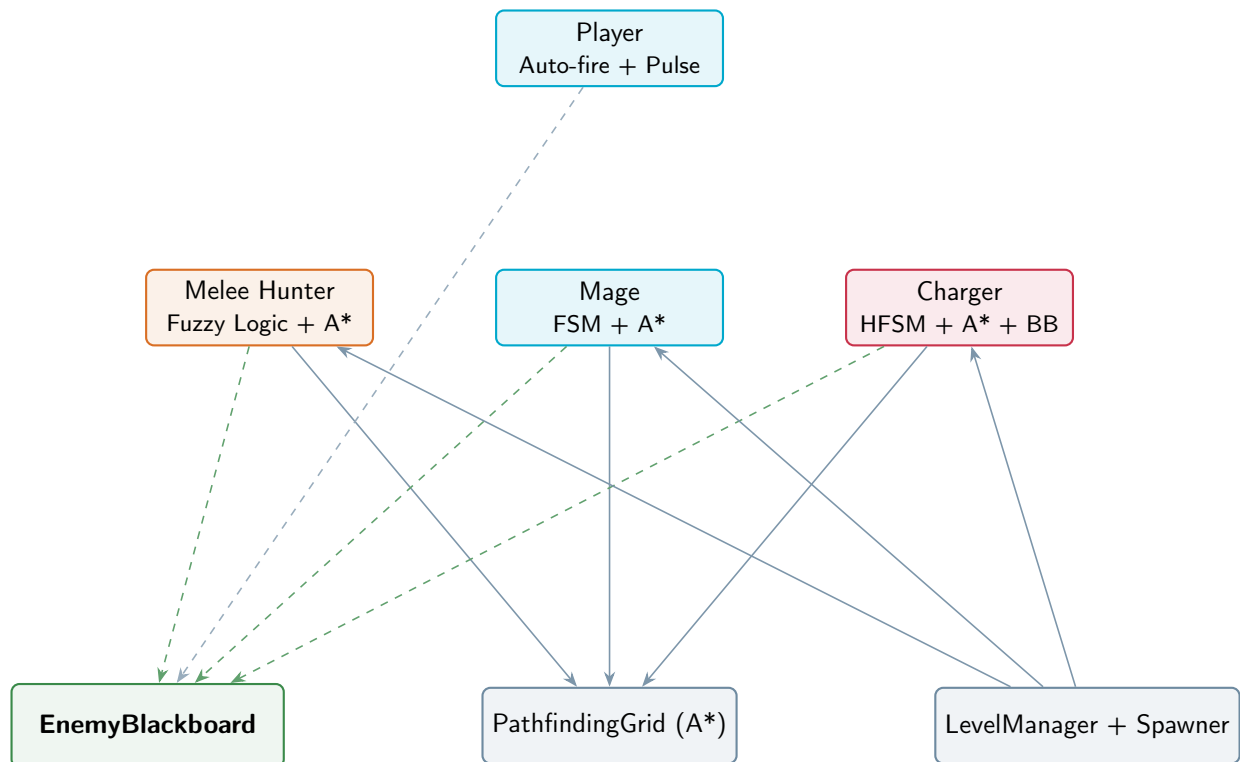


Figure 1: Top-level architecture. Solid arrows are direct usage; dashed green arrows are read-s/writes against the shared Blackboard. The Spawner and LevelManager produce enemies; the PathfindingGrid is a passive service every agent queries.

3 Game overview: the player loop

Before diving into the AI, here is exactly what the player does and what the game does back. This grounds every later design decision.

3.1 The 30-second loop

1. Player spawns in the center of an arena (Storm Breaker, mage sprite).
2. Enemies spawn from the edges, around the player but never on top of them (we'll see how the spawner enforces this in 10).
3. The player moves with WASD. There is *no manual attack input*.

4. The mage automatically throws fireballs at the closest enemy and automatically detonates an Arcane Pulse around itself when threats get too close.
5. Killing enemies fills a wave quota. Finish the quota $\Rightarrow$ next wave. Five waves $\Rightarrow$ next “level” (difficulty tier). Three levels and the run is won.
6. Between waves the player gets stronger; between levels the player gets a permanent boost.

3.2 Why no attack button?

This was the single biggest combat-design decision. We wanted the player’s attention 100% on *positioning*, because positioning is where the AI’s behavior becomes legible: you can only appreciate a Charger telegraphing its dash if you are not also worrying about pressing X at the right time. By making attacks automatic, the game becomes a *reading game*: “where do I stand so the Hunter can’t catch me, the Mage can’t line a shot, and the Charger can’t ambush me?”

This is also why the Arcane Pulse is automatic and defensive only: giving the player a panic button makes them *think* about positioning even more, because the pulse only fires when at least two enemies are inside 2.5 units. If you let an enemy get that close, the game punishes *and* rescues you in the same beat.

4 The player character

4.1 Stats and progression

Stat	Base value	Wave update
Max HP	100	+10 on level-up (full heal); +20 partial heal between waves
Movement speed	3 u/s	+0.3 on level-up (cap 6 u/s)
Fire rate	0.65 s	$\times 0.9$ each wave (floor 0.25 s)
Fireball damage	10	+2 each wave
Arcane Pulse	20 dmg, 2.5 u	— (constant)
Pulse cooldown	4 s	— (constant)

Table 1: Player stats and how they grow. The progression is multiplicative on fire rate (so the curve is gentle, not a cliff) and additive on damage and HP.

4.2 Auto-aim fireball

Every `fireRate` seconds, the player asks the world for the closest *living* enemy and spits a homing fireball at it. “Living” is non-trivial: a dead enemy still exists for a fraction of a second while its death animation plays. If we didn’t filter out dead enemies, fireballs would sometimes track corpses, which looks dumb. So the closest-enemy search calls `IsDead()` on each candidate.

```

1 Transform GetClosestEnemy()
2 {
3     GameObject[] enemies = GameObject.FindGameObjectsWithTag("Enemy");
4     Transform closest = null;
5     float minDist = Mathf.Infinity;
6
7     foreach (GameObject enemy in enemies)
8     {
9         // Filter dead enemies so the fireball doesn't track corpses.
10        MeleeHunter mh = enemy.GetComponent<MeleeHunter>();
11        if (mh != null && mh.IsDead()) continue;
12        Charger c = enemy.GetComponent<Charger>();
13        if (c != null && c.IsDead()) continue;
14    }

```

```

15         float dist = Vector3.Distance(enemy.transform.position, transform.
16             position);
17         if (dist < minDist) { closest = enemy.transform; minDist = dist; }
18     }
19     return closest;

```

Listing 1: Fireball auto-targeting (Player.cs).

The fireball itself (`fireball.cs`) keeps a fallback direction in case its target dies mid-flight: instead of freezing in mid-air it keeps flying in the last known direction. Tiny detail, but the alternative (fireballs hanging in space) was painful to look at during playtests.

4.3 Arcane Pulse (automatic defensive AoE)

The Arcane Pulse is the only “ability” in the game and we had a lot of internal debate about it. Original idea: a manual button. We dropped that for two reasons:

- It clashed with the “no attack button” rule.
- It was too good when the player remembered to press it, and useless when they panicked.

The final design fires the pulse *only* when there are at least two enemies inside 2.5 units. That way it is a punishment-and-reward in one: “you let multiple enemies surround you, here is your bailout, but you have to actually be in danger to get it.” Cooldown is 4 s, so spamming the player with single enemies still works as a strategy for the AI.

```

1 if (Time.time >= _nextPulseTime &&
2     CountEnemiesInPulseRange() >= minEnemiesForPulse)
3 {
4     PerformPulse();
5     _nextPulseTime = Time.time + pulseCooldown;
6 }

```

Listing 2: Pulse trigger (Player.cs).

Cut: knockback. Earlier versions of the pulse pushed enemies back. This looked great with Hunters but *terrible* with Chargers, because a Charger mid-dash got launched backwards and skated across the arena like it was on ice. We removed knockback entirely; the pulse is now pure damage.

5 Enemy archetype 1: Melee Hunter (Fuzzy Logic)

5.1 Role

The Hunter is the *adaptive pursuer*: the most common, weakest unit, but its speed is not a constant. It slows down or speeds up based on how the fight is going — low HP makes it cautious, low player HP makes it bloodthirsty.

5.2 Why Fuzzy Logic for this enemy

We considered three options for the Hunter’s speed:

1. Constant speed.
2. Hard if/else thresholds (“if HP < 50% then slow”).
3. Fuzzy logic with linguistic membership (Healthy / Wounded for both the enemy and the player).

Constant speed is boring. If/else thresholds create that distinctive “video-game enemy” vibe where you can see the seams: at HP = 50% the enemy is fast, at HP = 49% it is suddenly slow. Fuzzy logic gives a *smooth* blend: a Hunter at 30% HP and 70% player HP is mostly CAUTIOUS but a little NORMAL, and the resulting speed is somewhere between the two. The transition reads as the enemy *deciding* rather than flipping a flag.

5.3 Membership functions

We use the simplest possible setup — linear membership — because complexity buys you nothing here. The Hunter’s HP gives two fuzzy sets: HEALTHY (membership $\mu_H = h$) and WOUNDED ($\mu_W = 1 - h$), where $h \in [0, 1]$ is the normalized HP. Same idea for the player: HEALTHYPJ ($\mu_{HP} = p$) and WOUNDEDPJ ($\mu_{WP} = 1 - p$).

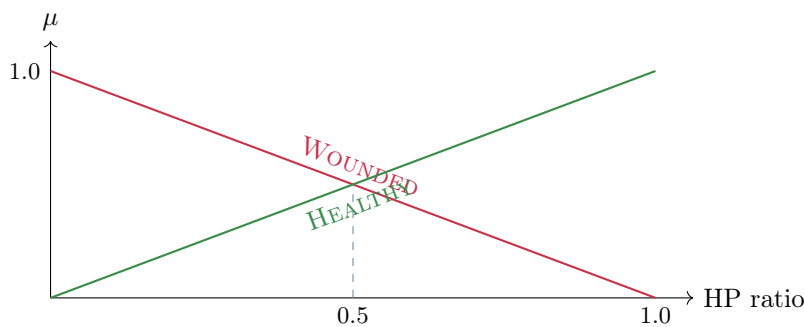


Figure 2: Linear fuzzification of HP into the HEALTHY / WOUNDED sets. At 50% HP, both sets fire equally ($\mu = 0.5$).

5.4 Rule base (Mamdani-style)

Four rules, all firing at once. Each combines one Hunter set with one player set, using **min** as the fuzzy AND:

Hunter HP	Player HP	$\Rightarrow$ Speed mode	Output (u/s)
Wounded	Healthy	CAUTIOUS	0.8
Wounded	Wounded	NORMAL	2.0
Healthy	Healthy	AGGRESSIVE	3.0
Healthy	Wounded	FRANTIC	5.0

The rationale is human-readable, which is half the point of fuzzy logic: a wounded enemy facing a fresh player retreats (cautious); a healthy enemy facing a dying player goes for the kill (frantic). We defuzzify with a weighted-average centroid:

$$\text{speed} = \frac{w_{\text{Cau}} \cdot 0.8 + w_{\text{Nor}} \cdot 2.0 + w_{\text{Agr}} \cdot 3.0 + w_{\text{Fre}} \cdot 5.0}{w_{\text{Cau}} + w_{\text{Nor}} + w_{\text{Agr}} + w_{\text{Fre}}}$$

where each w is the firing strength of the corresponding rule. The result is a single, smooth speed value that interpolates the four modes in real time.

5.5 Bonus: Blackboard read

There is one place where the Hunter peeks at the Blackboard: if the player is currently stunned (the Charger just hit them), the Hunter multiplies its computed speed by **stunnedBonus** (1.5) to rush in for the kill. This is the simplest possible piece of emergent teamwork — a single boolean flag — and it transforms the feel of the fight, because suddenly being stunned is *lethal*.

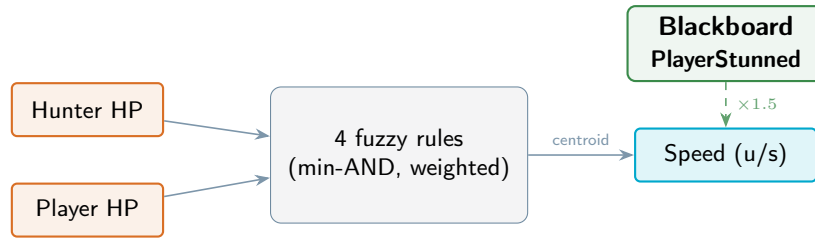


Figure 3: Hunter speed pipeline: fuzzification of two HP signals, four-rule inference, weighted-centroid defuzzification, optional Blackboard multiplier.

6 Enemy archetype 2: Charger (Hierarchical FSM + Blackboard)

6.1 Role

The Charger is the *shock weapon*: low frequency, very high impact. It rams the player in a straight line, deals a chunk of HP and stuns them for a second. After it hits (or misses), it is *itself* stunned and takes double damage — the only enemy in the game with a proper risk/reward window.

6.2 Why an HFSM (and not a flat one)?

A flat FSM works, but you end up duplicating the “what happens if this enemy dies” logic in every state. The Charger has many states (Wander, Approach, Prepare, Charge, Stunned), and any of them can be interrupted by death. Two-level HFSM:

- **TopState**: Alive | Dead.
- **AliveState** (only ticks if TopState = Alive): Wander → Approach → Prepare → Charge → Stunned.

When HP reaches zero we flip TopState to Dead *once*, and every sub-state stops being evaluated. No `if (isDead) return;` sprinkled through five different methods.

Top: Alive

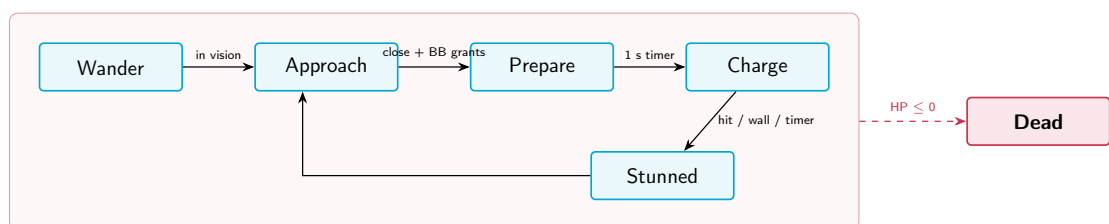


Figure 4: Charger HFSM. The big rounded box is the **Alive** top state; everything inside it is sub-FSM. The dashed red transition is the top-level override — it fires from *any* sub-state.

6.3 Why telegraph the charge with a 1-second pause?

The **Prepare** state does nothing useful, mechanically: the Charger just stops for 1 second before launching. That’s the whole point. Without the pause, the Charger feels like a homing missile: cheap and unfair. With the pause, the player has a *window to react* — sidestep, use a wall, bait the dash. The same algorithm goes from “unfair” to “fair, hard, and fun” just by adding a visible wind-up. This is the “believability over optimality” rule in action.

6.4 Why the Charger takes $\times 2$ damage while Stunned

The **Stunned** state is the dual of **Prepare**. Once the charge resolves, the Charger is immobile and vulnerable for 1 s. Multiplying incoming damage by 2 turns this into the *combat rhythm*: the player baits the dash, sidesteps, then unloads while the Charger is recovering. Without the multiplier, a Charger costs as much to kill mid-charge as it does mid-stun, and the entire wind-up dance becomes pointless.

6.5 Coordination via Blackboard

If three Chargers spawn at once and all dash simultaneously, the arena turns into a pinball machine and the player has nothing to read. We solved this with the simplest possible piece of multi-agent coordination: a counter on the Blackboard.

```

1 public static int MaxSimultaneousCharges = 1;
2 private static int _activeCharges;
3
4 public static bool RequestCharge()
5 {
6     if (_activeCharges < MaxSimultaneousCharges)
7     { _activeCharges++; return true; }
8     return false;
9 }
10
11 public static void ReleaseCharge() =>
12     _activeCharges = Mathf.Max(0, _activeCharges - 1);

```

Listing 3: Charge arbitration (`EnemyBlackboard.cs`).

A Charger that wants to enter **Prepare** must *ask* the Blackboard. If another Charger is already charging, the request is denied and the second one stays in **Approach** until its turn. This is a Blackboard pattern at its purest — agents don’t know about each other, they just talk to a shared store. From the player’s point of view, it looks like the Chargers are taking turns. They’re not: they’re each minding their own state machine and reading a counter.

6.6 Stun handshake with the player

When the Charge connects, the Charger:

1. Calls `Player.TakeDamage(15)` and `Player.Stun(1s)`.
2. Writes `Blackboard.SetPlayerStunned(1s)`.
3. Transitions itself to **Stunned**.

At the same time, every Hunter in the arena is reading `Blackboard.PlayerIsStunned` as part of its fuzzy speed calculation. So a single Charger hit triggers an arena-wide rush without any direct call between agents. *This* is the moment where the Blackboard pattern earns its keep — the one design decision the rest of the game is built around.

7 Enemy archetype 3: Mage / Ranged Shooter (FSM)

7.1 Role

The Mage is the *cold sniper*. It hangs back, fires a slow but hard-hitting projectile, and runs away if you close the gap. The other two enemies want you melted; the Mage wants you in a specific *distance band*.

7.2 The three-state FSM

Originally the Mage had four states (Idle, Chase, Attack, Retreat) and an **Idle** mode that activated when the player was outside its detection radius. We **cut Idle entirely** during playtests: as soon as the player ran a few steps away, the Mage stopped moving and turned into a soft, free target you could pick off with fireballs. Removing Idle means the Mage is *always* aware of the player — it spawns and starts walking immediately. The current FSM is:

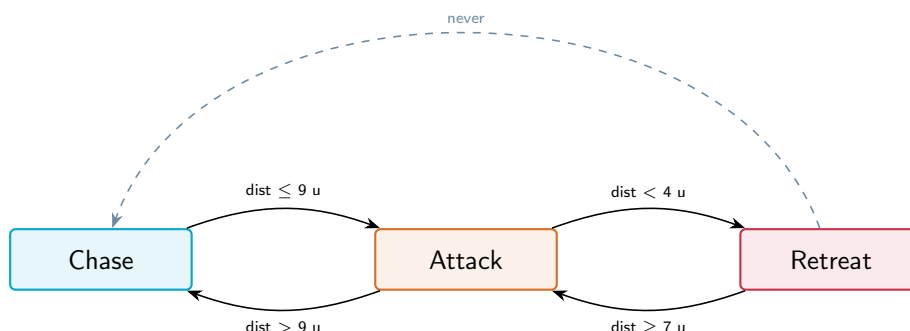


Figure 5: Mage FSM. The dashed line shows the absent transition: the Mage never re-enters Chase from Retreat; once it opens a window, it shoots from Retreat too.

7.3 Why the Mage shoots while retreating

In **Retreat** the Mage is moving away from the player, but it *still* fires its projectile on cooldown. This was deliberate. Otherwise, a player who knows the Mage’s threshold can just walk into 4 u, wait for the Mage to safely retreat, then walk in again. By shooting during retreat, the Mage punishes you for being aggressive *and* for being passive. There is no safe distance — only a preferred one.

7.4 The three-band distance system

The Mage cares about three radii:

- **attackRange** = 9 u — inside this, it can fire.
- **preferredRange** = 7 u — the band it tries to maintain.
- **retreatDistance** = 4 u — inside this, it runs away.

In **Attack** mode, the Mage uses A* to micro-adjust its position toward the preferred range, with a 1.2 u tolerance dead-band so it doesn’t oscillate. The result is an enemy that “circles” the player at a comfortable shooting distance without us writing any circling behavior — it falls out of the simple band.

8 Pathfinding: a custom A*

8.1 Why we wrote our own A* instead of using NavMesh

Unity’s NavMesh is overkill for a 2D grid game and a pain to set up for tilemaps. We rolled a tiny grid-based A* (`PathfindingGrid.cs`) that:

1. Bakes the world once at startup using `Physics2D.OverlapCircle` against an obstacle layer to mark each grid cell as walkable or not.

2. Exposes `FindPath(start, end)` as a synchronous query.
3. Exposes `GetSpawnCandidates(center, minDist, maxDist)` so the Spawner can place enemies on legal tiles (see 10).

8.2 Cost function and heuristic

We use the classic

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the path cost so far and $h(n)$ is the Chebyshev-style diagonal heuristic:

$$h(a, b) = \begin{cases} 14 \cdot dy + 10 \cdot (dx - dy) & \text{if } dx > dy, \\ 14 \cdot dx + 10 \cdot (dy - dx) & \text{otherwise.} \end{cases}$$

The 10/14 weights come from the integer approximation of $1/\sqrt{2}$ at scale 10 — diagonal moves cost a bit more than orthogonal ones, but not enough to forbid them. This is the standard trick from grid pathfinding tutorials and it gives nice-looking paths without rounding artifacts.

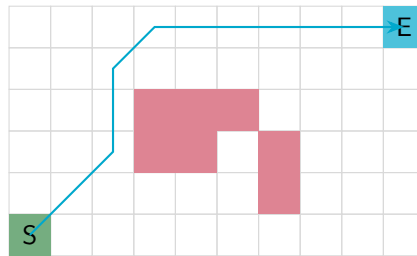


Figure 6: A* in action over a baked grid. Red cells are obstacles, green is the start, cyan is the goal. Diagonal moves are allowed and weighted 14 vs. 10 for orthogonal moves.

8.3 Why every enemy queries A*, every 0.3–0.5 seconds

We don't recompute the path every frame — that would be wasteful and also *look wrong* (enemies twitching as the path flickers). Each enemy has its own `pathUpdateInterval` (Charger 0.3s, Hunter and Mage 0.4s) and follows the cached waypoints in between. When an enemy reaches a waypoint within 0.35 u, it advances to the next one. If the path runs out, it falls back to direct steering toward the target.

Why the Charger refreshes faster. Because it switches between **Approach** and **Stunned** multiple times per kill, and re-entering **Approach** after a stun forces an immediate path recalculation (`_pathTimer = 0`). Otherwise the Charger would lazily follow a stale path from before the stun, walking into walls.

9 The Blackboard: small, static, surprisingly load-bearing

9.1 Why a static class instead of a singleton

We considered three options: a singleton `MonoBehaviour`, an interface injected into each enemy, or a plain `static class`. We picked the static class because:

- There is, by definition, exactly one Blackboard.
- Enemies don't need to find it (`FindObjectOfType` would run on every spawn).
- The state lives *outside* the scene graph, so reloading the scene doesn't accidentally duplicate it.

9.2 What the Blackboard actually holds

Surprisingly little. The whole API fits on one slide:

Field / method	Type	Purpose
PlayerPosition	Vector2	latest seen player coords
PlayerHealthRatio	float	for fuzzy rules
PlayerIsStunned	bool	speeds up Hunters when true
RequestCharge()	bool	arbiter, max 1 active
ReleaseCharge()	–	frees the slot
Tick()	–	decays the stun timer

9.3 Emergent coordination

The Blackboard wires up two pieces of behavior that look authored but aren't:

1. **Stun-rush.** The Charger stuns the player; Hunters react.
2. **Turn-based dashes.** Multiple Chargers “negotiate” so only one dashes at a time.

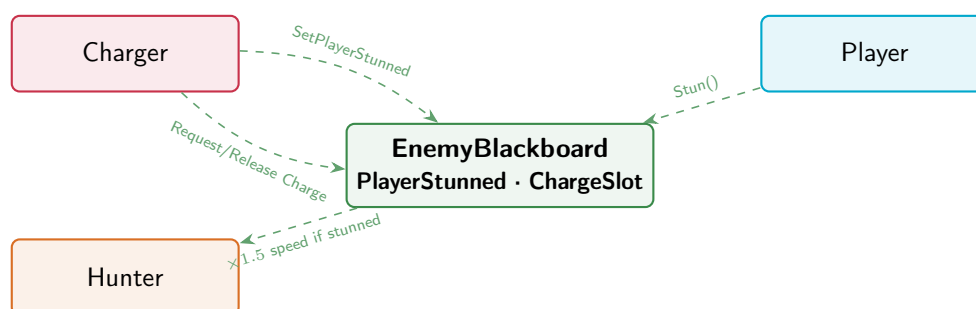


Figure 7: Blackboard data flow. Note that no agent talks to another agent directly — everything is mediated by the central store.

10 Spawning: where do enemies appear, and why

The `EnemySpawner` is dumb on purpose: every `spawnRate` seconds it picks a position from the grid's `GetSpawnCandidates` and instantiates an enemy. The cleverness is in how candidates are chosen:

- Only walkable cells.
- Inside an annulus around the player: at least `minSpawnDistance` (7 u) but no more than `maxSpawnDistance` (14 u).

The annulus matters. If we let enemies spawn inside the player's pulse radius, the game becomes “stand still and farm.” If we let them spawn across the whole arena, half of them never reach the player before dying and the wave feels empty. 7–14 u is the sweet spot we found in playtests — close enough to be threatening, far enough to feel fair.

There's a hard cap (`maxEnemies` = 15 by default) for the same reason: more than 15 active enemies on screen and the player can't read individual behaviors anymore. Past that point, no amount of clever AI matters — it all looks like soup.

11 Wave / level progression

11.1 Structure

- **Level** = 5 waves.
- **Game** = 3 levels.
- Per-wave kill quota: {3, 6, 10, 15, 20}.
- Per-wave spawn rate: {3, 2.5, 2, 1.5, 1} seconds.

11.2 Why two different curves?

Increasing the kill quota *and* reducing the spawn rate at the same time is more interesting than either alone:

- Bigger quota alone $\Rightarrow$ longer, identical-feeling waves.
- Faster spawn alone $\Rightarrow$ chaos with no goal.
- Both at once $\Rightarrow$ the arena fills up faster *and* stays full longer. Each wave is denser than the last, in two independent dimensions.

11.3 When the player gets stronger

Every time a wave clears, `ApplyWaveProgression` runs:

```

1 public void ApplyWaveProgression(int wave, int level)
2 {
3     fireRate      = Mathf.Max(0.25f, fireRate * 0.90f);
4     fireballDamage += 2;
5
6     if (wave == 0) // i.e. just leveled up
7     {
8         speed      = Mathf.Min(6f, speed + 0.3f);
9         maxHealth += 10;
10        currentHealth = maxHealth; // full heal
11    }
12    else
13    {
14        currentHealth = Mathf.Min(maxHealth, currentHealth + 20);
15    }
16 }
```

Listing 4: Player progression hook (`Player.cs`).

The asymmetry is intentional: between waves you get a small heal (“thanks for clearing that one”); between levels you get a major upgrade and a full heal (“okay, real reward”). It mirrors the roguelite “checkpoint” pacing without actual checkpoints.

12 UI / UX

The HUD is intentionally minimal — four widgets and nothing else:

- **Player health bar.** Slider whose color interpolates from green at full HP, through yellow at 50%, to red near zero (see `PlayerHealthBar.cs`). The bar tracks the player’s `maxHealth` live, so it updates the moment a level-up bumps the cap.

- **Per-enemy health bars.** Each enemy has its own slider child object. We tried hiding them and the fight became unreadable — you couldn’t tell whether a Charger was about to die or had just spawned. Visible bars + $\times 2$ -damage feedback on a Stunned Charger create a tight feedback loop: hit, see bar drop, predict the kill.
- **Wave / Level text.** TextMeshPro labels updated by `LevelManager.RefreshUI()`. “WAVE 3/5” and “LEVEL 2”.
- **Main menu.** Play, Options, Exit (`MainMenu.cs`). Loads the `Game` scene on Play.

A specific UI detail worth documenting: when the player is at exactly `maxHealth`, Unity’s Slider leaves a one-pixel gap at the right side because of the handle. We force `fill.fillAmount = 1f` in that edge case, otherwise it looks like the player is missing 1 HP at all times.

13 What we cut, and why

This is the part that’s missing from most game reports. Cuts are *decisions*, and explaining them is half the design rationale.

- **The Tomes upgrade system from the GDD.** A run was supposed to involve picking magical Tomes (Fire, Ice, Lightning, Melee) at level-up. We dropped it because (a) it’s a roguelite meta-system that needs balancing across hundreds of runs to feel good, and (b) it would have buried the AI under a pile of build decisions. The current progression is fewer, sharper levers.
- **Manual attack and dash.** Both were in the GDD. Both pulled attention away from positioning. See 4.
- **Knockback on the Arcane Pulse.** See 4 — it broke the Charger.
- **The Mage’s Idle state.** See 7 — it turned the Mage into a free target.
- **12 stages, 3 biomes, bosses, meta-currency.** All ambitious GDD bullets. Cut for scope. The current 3-level / 5-wave structure is enough to demonstrate the AI; padding it with more arenas would have been make-work.
- **XP orbs and pickups.** Replaced by the simpler “kill quota fills, wave advances” loop. Same effect, no extra systems to balance.

The pattern is the same every time: “does this thing make the AI read more clearly?” If not, it got cut.

14 Reflection & takeaways

14.1 What worked

One technique per role. Picking exactly one “hero” AI technique per enemy — Fuzzy for Hunter, HFSM for Charger, FSM for Mage — kept each script understandable. We never had to debate “should the Hunter also be an HFSM?” because the role made the technique obvious.

Believability beats optimality, every time. The Charger `Prepare` pause and the Mage’s `Retreat`-while-shooting are the two moments players most often comment on (“oh, I get it!”), and both are deliberately *sub-optimal* from the agent’s POV. That is the whole pedagogical point of the project, and we got to see it work in playtests.

Blackboard > direct messaging. Two emergent behaviors (stun-rush, turn-based dashes) cost us a single static class with 15 lines of state. If we had wired Hunter $\leftrightarrow$ Charger directly, the code would have doubled and we'd be debugging null references every time an enemy died mid-broadcast.

14.2 What we'd do differently

Test fuzzy speeds with a graph, not by feel. For most of development, “does the Hunter feel right?” was tuned by playing. A small editor tool that plotted the defuzzified speed across the (h, p) plane would have saved hours.

Less aggressive A* updates. 0.3–0.5 s is fine for the final version, but during prototyping we ran A* every frame on every enemy and the framerate cratered. We learned to budget pathfinding the hard way.

Document the Blackboard contract earlier. The static class grew organically; a few times we had two enemies disagreeing about whether `PlayerIsStunned` was authoritative or a hint. Writing the table from 9 on day one, not month three, would have helped.

14.3 The thesis, one more time

We started with a simple hypothesis: *enemies that look intelligent are not the ones running the most expensive algorithms; they are the ones whose simple algorithms are wrapped in good design.* StormBonk is small, on purpose. The Hunter's fuzzy rules are four lines. The Charger's HFSM is two enums. The Blackboard is six fields. And yet, when a Charger telegraphs a dash, hits, stuns the player, and two Hunters break into a sprint to finish the kill — nobody scripted that. It's the four techniques colliding under their own rules.

That's the design we wanted to build. That's what's in the build.

A File map

File	Role	Notes
<code>Player.cs</code>	Player	Movement, auto-fire, pulse, progression, stun.
<code>fireball.cs</code>	Projectile	Homing fireball with fallback direction.
<code>MeleeHunter.cs</code>	Enemy (Fuzzy)	4-rule fuzzy speed + A* path.
<code>Charger.cs</code>	Enemy (HFSM)	TopState/AliveState, Blackboard arbiter.
<code>Prefabs/MageEnemy.cs</code>	Enemy (FSM)	Chase/Attack/Retreat with band tolerance.
<code>MageProjectile.cs</code>	Projectile	Mage's slow heavy shot.
<code>EnemyBlackboard.cs</code>	Service	Static. Player stun + charge arbiter.
<code>PathfindingGrid.cs</code>	Service	Custom grid A*, baked at start.
<code>EnemySpawner.cs</code>	Service	Annular spawn around the player.
<code>LevelManager.cs</code>	Manager	Wave/level structure + UI updates.
<code>PlayerHealthBar.cs</code>	UI	Color-interpolated HP slider.
<code>MainMenu.cs</code>	UI	Main menu / scene loader.

B Tuning constants (current build)

System	Constant	Value
Player	maxHealth	100
Player	speed	3 u/s (cap 6)
Player	fireRate	0.65 s (floor 0.25)
Player	fireballDamage	10
Player	pulseRadius	2.5 u
Player	pulseDamage	20
Player	pulseCooldown	4 s
Player	minEnemiesForPulse	2
Hunter	speeds (Cau/Nor/Agr/Fre)	0.8 / 2.0 / 3.0 / 5.0
Hunter	stunnedBonus	×1.5
Charger	maxHealth	40
Charger	prepareTime	1 s
Charger	chargeDuration	3 s
Charger	chargeSpeed	7 u/s
Charger	stunDuration	1 s
Charger	stunDamageMultiplier	×2
Charger	playerStunDuration	1 s
Mage	attackRange	9 u
Mage	preferredRange	7 u
Mage	retreatDistance	4 u
Mage	attackCooldown	2.5 s
Spawner	maxEnemies	15
Spawner	minSpawnDistance	7 u
Spawner	maxSpawnDistance	14 u
A*	node radius	0.5 u
A*	ortho/diag cost	10 / 14
LevelManager	waves per level	5
LevelManager	total levels	3
LevelManager	kill quotas	3, 6, 10, 15, 20
LevelManager	spawn rates (s)	3, 2.5, 2, 1.5, 1